Pyramide de tests — AgentInsurFlow

Reprise de la règle `testing-pyramid` de Sorabel (niveaux, critères, outillage), ***plus*** *les proportions décidées au sprint 3 (S3-22) et leur porte.*

Portée : `rag`, `serveur-mcp`, `agentic-cluster`, `fonction-calcul` (Python), `underwriting-admin` (C#) et `e2e/`. Un nouveau dossier `*/tests/unit` ou `*/tests/integration` non déclaré dans `tools/pyramide.py` fait échouer `tests/unit/test_pyramide.py`. `tests/` (racine : doc et outils) n'est pas un service et n'est pas compté. La **gateway** est comptée à part (forme diamant, voir plus bas).

Les niveaux et leur part

Niveau	Part visée	Critère d'appartenance	Python (`rag`, `serveur-mcp`, `agentic-cluster`, `fonction-calcul`)	C# (`underwriting-admin`)	Cible
TU — unitaire	**70 %**	Aucune I/O, aucun framework, aucun conteneur. Domaine et cas d'usage, **ports simulés** seulement. Millisecondes.	`rag/tests/unit` (`pytest`, `tests/fakes.py`)	`*.Domain.Tests`, `*.Application.Tests` (xUnit)	`make test`
TI — intégration	**20 %**	Un adapter face à sa vraie dépendance ou à un double fidèle (PostgreSQL du compose, `httpx.MockTransport`, WireMock.Net), **ou** l'API complète en mémoire (contrat HTTP : routes, codes, ProblemDetails).	`rag/tests/integration` (`TestClient`, `httpx.AsyncClient`)	`*.Infrastructure.Tests`, `*.Api.Tests` (`WebApplicationFactory`)	`make test`, `make test-integration`
E2E	**10 %**	La solution qui tourne (compose ou staging), **via la gateway**, avec ses vraies dépendances. Lent, exclu du cycle rapide.	`e2e/` (pytest, `E2E_URL`)	—	`make test-e2e URL=…`

- **Porte** : `make pyramide` (`tools/pyramide.py`) compte les tests de chaque niveau et **échoue si TU < 65 %, TI > 30 %, E2E > 15 %, si un niveau est vide, ou si `dotnet test --list-tests` échoue**. Elle tourne en CI (job `pyramide`) et est **bloquante** (contexte exigé par `infra/protection-main.json`).
- **Pourquoi TI ≤ 30 % et non 25 % (#179, décision d'Acerola du 02/11/2026)** : jusqu'au 02/11, `VERSION` faisait échouer `dotnet test` sans erreur visible : les tests C# comptaient 0 et la porte affichait 24 % de TI au lieu de 31 %. Une fois le compte juste, les variations de règles ont été descendues en TU, jusqu'à

environ 27 %. Le reste, ce sont des contrats HTTP de services fins et des preuves sur le vrai PostgreSQL (droits, vues, BOLA), et le domaine est déjà couvert par les TU. Aller plus bas supposait de supprimer ces preuves ou d'écrire des TU redondants, ce que cette règle interdit. La part visée reste 20 % : c'est un écart assumé. Un nouveau TI se compense par un TU de domaine utile, pas par un nouveau relâchement du seuil.
- **Couverture ≥ 80 %** de lignes, par projet, mesurée sur TU + TI. Aucun quota par niveau.

TU ou TI ?

Un test qui touche un socket, un fichier, une base ou une horloge système **n'est pas unitaire**, quel que soit le nom de son dossier. Un TU ne simule que des **ports** (`domain/ports.py`, interfaces C# du Domain ou de l'Application), jamais un détail d'infrastructure. Exception assumée : un adaptateur HTTP testé avec `httpx.MockTransport` (en mémoire, sans socket) reste un TU.

TI ou E2E ?

Le TI démarre l'application **en mémoire**, backends doublés : il ne prouve rien du packaging ni de la configuration réelle. L'E2E démarre la solution **telle qu'elle est déployée**. Un changement qui touche le packaging, la configuration ou le démarrage n'est pas terminé sur la foi des seuls TU et TI.

La gateway : forme « diamant »

Proxy (routes, politiques d'autorisation et limites de débit déclaratives) : presque aucune logique isolable. Ses tests sont des **TI** (`WebApplicationFactory` devant WireMock.Net), comptés à part. Écrire des TU sur sa configuration pour gonfler un compteur est un anti-pattern.

Ce que Claude doit faire

- Avant d'écrire un test, nommer **son niveau** et vérifier qu'il en respecte le critère.
- Quand un niveau manque, le signaler plutôt que de compenser en gonflant un autre.
- Ne jamais écrire de test dont la seule fonction est d'augmenter la couverture ou un pourcentage.
- Viser la répartition par des **TU de domaine** (traitement du corpus, porte de pertinence, verrous, BOLA), pas par des TI supplémentaires.